

Introduction to Java with genAI

In this activity, we'll see some ways to use generative artificial intelligence (genAI) tools in this course, especially if you're new to Java.

Throughout the course, we will occasionally use genAI tools, being careful to still build foundational understanding and skills. In both education and industry, we want genAI tools to *support*, not undermine, our goals. We'll also discuss how genAI tools are currently used in industry, including best practices and ethical considerations.

You'll work in pairs today; next time, you'll work in larger teams. If you're new to Java, work with someone else who is new to Java. And if you already have some Java experience, find a partner with a similar level of Java experience.

Content Learning Targets

After completing this activity, you should be able to say:

- I can use genAI tools to learn about Java syntax and features.

Process Skill Goals

During the activity, you should make progress toward:

- Leveraging prior knowledge and experience of other students. (Teamwork)

Facilitation Notes

This activity is for the first day of class. Prereq: some programming experience, e.g., CSSE 120.



Copyright © 2026 Ian Ludden, based on [prior work](#) of Mayfield et al. This work is under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Part 1 Generative AI Tools for Software Development

Questions (20 min)

Start time:

If you are **new to Java**, complete #1. If you are **already familiar** with Java, complete #2.

1. GenAI tools can help you switch from another programming language to Java.
 - a) Open the provided `ConvertToJava.java` file.
 - b) Find a short function or code snippet (ideally, one you wrote) in a non-Java language with which you are familiar.
 - c) Have a GenAI tool (e.g., [Claude](#), [ChatGPT](#), [Gemini](#)) convert your code into Java ([sample](#)).
 - d) Test the Java version: copy/paste into `ConvertToJava.java` and modify the method call.
 - e) If the response doesn't explain the changes, ask. **What similarities and differences do you notice?** (Tip: Repeat for other non-Java code to help you adapt to Java syntax.)
 - f) When you finish this activity, take a look at `JavaExamples.java` and review.

Answers will vary. Coming from Python, some differences include Java's required variable type declarations, the use of curly braces and semicolons instead of whitespace, and the change in syntax for function/method headers.

2. GenAI tools can help you learn about different ways to solve a problem using Java.
 - a) Open `JavaExamples.java` and look through the provided example methods.
 - b) Choose one method, and try to solve the same problem in a different way using your current knowledge of Java. Write your new version as a separate method with a similar name, and test it by calling it from the main method. (If you have trouble coming up with a completely new approach, it's OK to just make a small change.)
 - c) Copy/paste the original method into a GenAI chat tool such as [Claude](#), [ChatGPT](#), or [Gemini](#), along with the prompt, "Give me different ways to implement this Java method." Review the responses. As time allows, repeat for other methods.
 - d) **What new Java features/syntax did you learn?** Jot down some notes below. Which alternative do you like the best, and why? (Tip: Repeat with other Java code you've written.)
 - e) When you finish this activity, take a look at the first homework assignment.

Answers will vary. GenAI tools might suggest ternary operators, switches, `StringBuilder`, enumerations, lambdas, etc., mostly inappropriate or overkill for the problems at hand. These are probably new if you're coming from AP Computer Science or similar.